

Object-Oriented Programming (OOP) for AI/ML & Data Analytics

Task Report

Name: Suresh

Employee Id : BR26PY9916A

Purpose: Python Advanced Foundations for AI/ML & Data Analytics

1. Class

Definition

A **Class** is a blueprint or template used to create objects.

A class contains:

- Variables (Data)
- Methods (Functions)
- **Example :**

```
class Student:
```

```
    def display(self):  
        print("Welcome to Python")
```

Real-World Example

Car Blueprint

|
v

Describes:

- Engine
- Color
- Model

But it is not an actual car.

2. Object

Definition

An **Object** is an instance of a class.

Objects are created from classes.

Example

```
class Student:
```

```
    pass
```

```
s1 = Student()
```

```
s2 = Student()
```

Real-World Example

Class = Car

Objects:

Tesla

BMW

Audi

3. Constructor (__init__)

Definition

A constructor is a special method that runs automatically when an object is created.

Python constructor:

```
class Student:
```

```
    def __init__(self):
```

```
        print("Constructor Called")
```

```
s1 = Student()
```

Real-World Example

Creating a Bank Account

Account Created

|
v

Initialize:

Account Number

Balance

Customer Name

4. Instance Variables

Definition

Variables created using self inside a constructor or method.

Each object gets its own copy.

Example

```
class Student:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
s1 = Student("Suresh")
```

```
s2 = Student("Ravi")
```

```
print(s1.name)
```

```
print(s2.name)
```

Real-World Example

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance
```

5. Class Variables

Definition

Variables declared directly inside a class.
Shared among all objects.

Example

```
class Student:
    college = "ABC College"
s1 = Student()
s2 = Student()
print(s1.college)
print(s2.college)
```

Difference: Instance Variable vs Class Variable

Instance Variable	Class Variable
Unique for each object	Shared by all objects
Defined using self	Defined directly in class
Stored separately	Stored once in class

6. Instance Methods

Definition

Methods that operate on object data.

Example

```
class Student:
    def __init__(self, name):
        self.name = name
    def display(self):
        print(self.name)
s1 = Student("Suresh")
s1.display()
```

7. Class Methods

Definition

Methods that work with class variables.

Example

class Student:

```
    college = "ABC College"
```

```
    @classmethod
```

```
    def show_college(cls):
```

```
        print(cls.college)
```

```
Student.show_college()
```

8. Static Methods

Definition

Methods that do not depend on:

- Instance Variables
- Class Variables

Example

class Calculator:

```
    @staticmethod
```

```
    def add(a, b):
```

```
        return a + b
```

```
print(
```

```
    Calculator.add(10,20)
```

```
)
```

9. self

Definition

`self` refers to the current object.

It is used to access instance variables and methods.

Example

class Student:

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def display(self):
```

```
        print(self.name)
```

Real-World Example

Self = Myself

Student Object

knows its own data.

10. cls

Definition

`cls` refers to the current class.
Used in class methods.

Example

```
class Student:

    college = "ABC"

    @classmethod
    def show(cls):
        print(cls.college)
```

Difference: self vs cls

self	cls
Refers to Object	Refers to Class
Used in Instance Methods	Used in Class Methods
Access Instance Variables	Access Class Variables

11. Object Lifecycle

Definition

The stages an object goes through from creation to destruction.
class Employe

Complete Real-World Example :

```
company = "Microsoft"    # Class Variable
def __init__(self, name, salary):
    self.name = name      # Instance Variable
    self.salary = salary
def display(self):        # Instance Method
    print(self.name, self.salary)
@classmethod
def show_company(cls):    # Class Method
    print(cls.company)
@staticmethod
def greet():              # Static Method
    print("Welcome")
```

```
e1 = Employee(  
    "Suresh",  
    50000  
)  
e1.display()  
Employee.show_company()  
Employee.greet()
```